



East West University

Lab 04: Digital Differential Analyzer (DDA) line drawing algorithm

Semester: Spring-2026

Department: Computer Science & Engineering

Course Title: Computer Graphics(CSE-420)

Section: 01

Submitted by:

Name	ID
Siam Ahmed	2022-2-60-074

Submitted to

Dr. Md. Tauhid Bin Iqbal

Assistant Professor

Department of Computer Science & Engineering

Date of submission: 16 February, 2026

```
#include <stdio.h>
```

```
#include <GL/gl.h>
```

```
#include <GL/glut.h>
```

```
#include <math.h>
```

```
float X1, Y1, X2, Y2, m;
```

```
float dx, dy;
```

```
void display(void)
```

```
{
```

```
    glClear(GL_COLOR_BUFFER_BIT);
```

```
    glColor3f(1.0, 1.0, 1.0);
```

```
    glPointSize(8.0f);
```

```
    glBegin(GL_POINTS);
```

```
    dx = X2 - X1;
```

```
    dy = Y2 - Y1;
```

```
    // Handle Vertical Line (Infinite Slope)
```

```
    if (dx == 0) {
```

```
        printf("--- Vertical Line ---\n");
```

```
        int steps = abs((int)dy);
```

```
        float yStep = (Y1 < Y2) ? 1 : -1;
```

```
        float currentY = Y1;
```

```
        for(int i = 0; i <= steps; i++) {
```

```
            float rX = roundf(X1);
```

```
            float rY = roundf(currentY);
```

```
            glVertex2f(rX / 100.0, rY / 100.0);
```

```

        printf("(%.2f, %.2f) -----> Rounded -----> (%.0f, %.0f)\n", X1, currentY, rX, rY);
        currentY += yStep;
    }
}
else {
    m = dy / dx;
    printf("--- Slope (m) = %.2f ---\n", m);

    float xInc = X1, yInc = Y1;
    int steps;
    float xStep, yStep;

    // Logic for different slope conditions
    if (m < -1) {
        // Case: m < -1 (Steep Negative)
        steps = abs((int)dy);
        yStep = (Y1 < Y2) ? 1 : -1;
        xStep = dx / (float)steps;
    }
    else if (m == -1) {
        // Case: m = -1 (Perfect Negative Diagonal)
        steps = abs((int)dx);
        xStep = (X1 < X2) ? 1 : -1;
        yStep = (Y1 < Y2) ? 1 : -1;
    }
    else if (m > -1 && m < 0) {
        // Case: m > -1 (Shallow Negative)
        steps = abs((int)dx);
        xStep = (X1 < X2) ? 1 : -1;
        yStep = dy / (float)steps;
    }
}

```

```

}
else if (m >= 0 && m <= 1) {
    // Case: Positive Shallow
    steps = abs((int)dx);
    xStep = (X1 < X2) ? 1 : -1;
    yStep = dy / (float)steps;
}
else {
    // Case: m > 1 (Positive Steep)
    steps = abs((int)dy);
    yStep = (Y1 < Y2) ? 1 : -1;
    xStep = dx / (float)steps;
}

// Execution and Printing
for(int i = 0; i <= steps; i++) {
    float rX = roundf(xInc);
    float rY = roundf(yInc);

    // Plotting scaled values
    glVertex2f(rX / 100.0, rY / 100.0);

    // Outputting the float coordinates and their roundf result
    printf("(%.2f, %.2f) -----> Rounded -----> (%.0f, %.0f)\n", xInc, yInc, rX, rY);

    xInc += xStep;
    yInc += yStep;
}
}

```

```

    glEnd();
    glFlush();
}

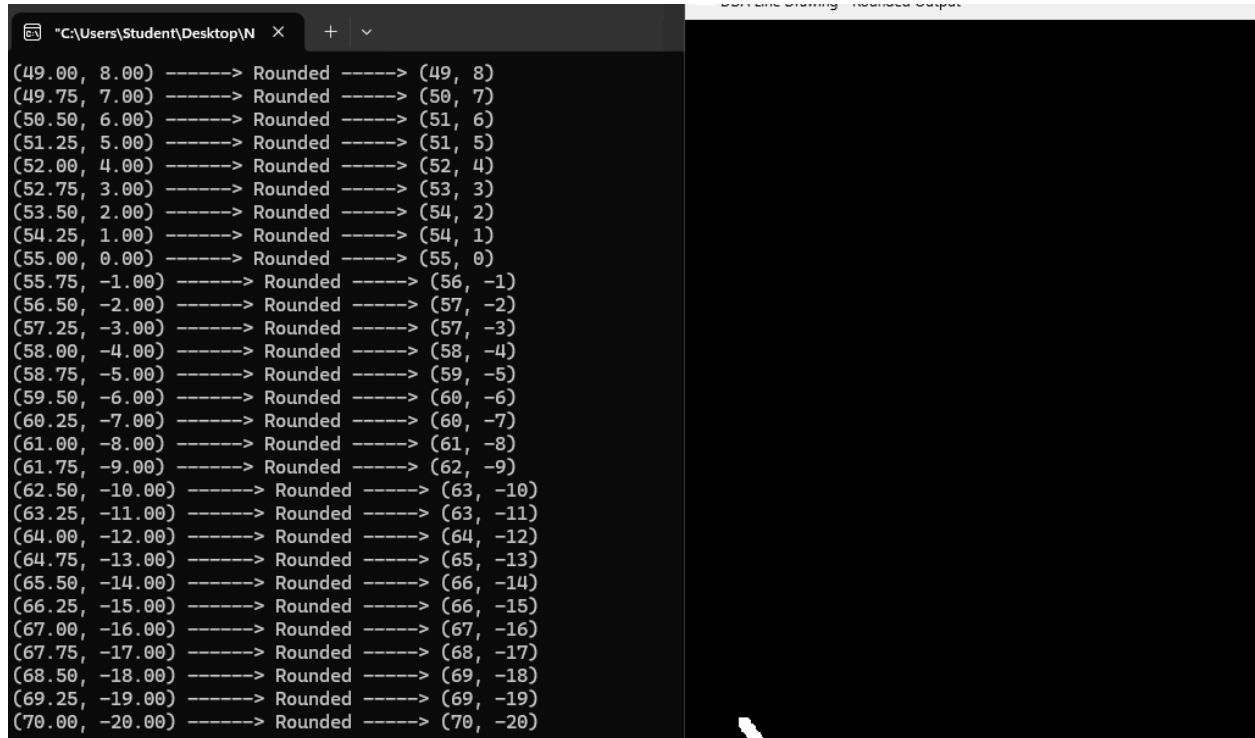
void init(void)
{
    glClearColor(0.0, 0.0, 0.0, 1.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    // Ortho is set to 5.0 to handle coordinates up to 500 comfortably
    glOrtho(0.0, 5.0, 0.0, 5.0, -1.0, 1.0);
}

int main(int argc, char** argv)
{
    printf("Enter (X1, Y1): ");
    scanf("%f %f", &X1, &Y1);
    printf("Enter (X2, Y2): ");
    scanf("%f %f", &X2, &Y2);
    printf("\nProcessing DDA Steps:\n");
    printf("-----\n");

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(600, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("DDA Line Drawing - Rounded Output");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
}

```

```
return 0;  
}
```



The screenshot shows a C++ program's output in a terminal window. The program generates a sequence of 20 pairs of coordinates, each pair consisting of a floating-point number and an integer, which are then rounded to the nearest integer. The output is displayed as a list of these rounded pairs, separated by spaces and arrows. The coordinates start at (49.00, 8.00) and increment by 0.25 in the x-value and -1.00 in the y-value for each subsequent pair, ending at (70.00, -20.00).

```
"C:\Users\Student\Desktop\N" X + v  
(49.00, 8.00) -----> Rounded -----> (49, 8)  
(49.75, 7.00) -----> Rounded -----> (50, 7)  
(50.50, 6.00) -----> Rounded -----> (51, 6)  
(51.25, 5.00) -----> Rounded -----> (51, 5)  
(52.00, 4.00) -----> Rounded -----> (52, 4)  
(52.75, 3.00) -----> Rounded -----> (53, 3)  
(53.50, 2.00) -----> Rounded -----> (54, 2)  
(54.25, 1.00) -----> Rounded -----> (54, 1)  
(55.00, 0.00) -----> Rounded -----> (55, 0)  
(55.75, -1.00) -----> Rounded -----> (56, -1)  
(56.50, -2.00) -----> Rounded -----> (57, -2)  
(57.25, -3.00) -----> Rounded -----> (57, -3)  
(58.00, -4.00) -----> Rounded -----> (58, -4)  
(58.75, -5.00) -----> Rounded -----> (59, -5)  
(59.50, -6.00) -----> Rounded -----> (60, -6)  
(60.25, -7.00) -----> Rounded -----> (60, -7)  
(61.00, -8.00) -----> Rounded -----> (61, -8)  
(61.75, -9.00) -----> Rounded -----> (62, -9)  
(62.50, -10.00) -----> Rounded -----> (63, -10)  
(63.25, -11.00) -----> Rounded -----> (63, -11)  
(64.00, -12.00) -----> Rounded -----> (64, -12)  
(64.75, -13.00) -----> Rounded -----> (65, -13)  
(65.50, -14.00) -----> Rounded -----> (66, -14)  
(66.25, -15.00) -----> Rounded -----> (66, -15)  
(67.00, -16.00) -----> Rounded -----> (67, -16)  
(67.75, -17.00) -----> Rounded -----> (68, -17)  
(68.50, -18.00) -----> Rounded -----> (69, -18)  
(69.25, -19.00) -----> Rounded -----> (69, -19)  
(70.00, -20.00) -----> Rounded -----> (70, -20)
```